

# Programming Language Design

## Module 2: Lambda Calculus

The foundation of functional programming

Slides © Grant Williams, [CC BY-SA 4.0](#).

This is an open educational resource.

Feel free to submit fixes, improvements, and new material [here](#).

# Last Module

We learned about how class is going to work

We learned what a formal language is.

We learned some programming language history.

We learned about programming paradigms.

We learned about one important paradigm in particular [which one]?

# This module

In this module, we'll be learning the fundamental model of computation behind functional programming: lambda calculus.

This has nothing to do with derivatives and limits, *calculus* is a generic term that refers to systems of calculation, and we are going to learn one.

Why? Let's talk about that

# Models of computation

What *is* a computer?

I mean, this is a computer \*points to lecture computer.

Many of you have laptops and tablets, I think we all agree that those are computers.

But what is the minimum thing that a device needs to be able to do to be called a computer?

Yes or no:

Is a rock a computer? Is an abacus a computer? Is a human a computer? Is a dog a computer? Is a bacterium a computer? Is a (biological) virus a computer? Is a car a computer? Is a tree a computer? Is an electron a computer? Is RAM by itself a computer? Is a CPU by itself a computer?

# My thoughts:

A rock is not a computer.

An abacus is not a computer.

A human *is* a computer (and a human *with* an abacus is certainly a computer).

A dog can be trained to compute.

A single bacterium can be a computer (DNA computer).

A virus probably too for the same reason.

A car is not a computer, but it has computers in it, so I guess if you include those.

A tree can respond to stimuli in different ways, so there's probably a way to make it compute, but I don't know how to do it.

An electron is not a computer.

RAM by itself is not a computer.

A CPU by itself is a computer if it has registers or some feedback mechanism.

# Okay, but what *is* a computer?

Broadly speaking, something that can carry out instructions.

Specifically, an **effective method**.

But if we want to be more formal, we need to actually specify a model of computation.

A model of computation is a specification for how to carry out computations.

You already know a model of computation because you have taken automata theory (or theory of computation).

[What is it?]

# The turing machine

The good old turing machine.

The instructions for a turing machine are encoded as states and transitions.

We can encode pretty much any practical computer program as a turing machine, as long as we can agree on how to represent things like IO.

For example, if we agree that cells 10000 through 20000 represent the colors of pixels (and the alphabet is, say, 32-bit integers), we can code a game for a 100 \* 100 screen and "run" it on a Turing machine.

## The turing machine (2)

In fact, the turing machine is the model of computation that underlies every imperative programming language that I am aware of.

Every time you modify a variable (or perform I/O), you are changing state.

Any time you have a conditional or loop, you are making a conditional transition.

Even though you aren't programming a turing machine directly, the design of the language is influenced by one. We think in terms of statements.

Every "statement" modifies state. We run code to move the computer from one state to another. Every program basically involves creating some kind of effect.



Questions?

# A different option?

What if I told you, that's not the only option?

There is a *completely different* model of computation that works in a fundamentally different way.

And yet, it is isomorphic to turing machines. That is, every turing machine can be converted into an expression in this language and back again without losing any information.

This language is called lambda calculus.

It was invented by [Alonzo Church](#) who was Alan Turing's Ph.D. advisor.

# Lambda calculus

Lambda calculus is based around the concept of a lambda function.

You may already be familiar with lambda functions from programming languages like Python, although their use of the term lambda is slightly incorrect.

A lambda function is basically a function that takes one argument and has one return value. That's it.

It turns out defining and applying functions of one argument is all you need to compute every computable function.

That seems impossible. Don't we need like...at least to be able to take as many arguments as we want? It turns out, we can get that for free.

# Lambda calculus

Lambda is a formal language for expressing computations (functions).

It actually can be a programming language, although just like a turing machine, you probably don't want to actually program in it.

It can express any computation a turing machine can express and vice versa. Any program you have ever written can be expressed as an expression in lambda calculus if you can agree on how to represent IO (same deal as turing machines). That is why it is called a model of computation.

Its main use is being a system that is so simple, you can't really remove a rule without breaking its turing equivalence.

So let's see the rules...

# Rules of lambda calculus

An expression is a valid *lambda term* if it matches one of these patterns

1.  $x$ : a variable by itself is a term. It doesn't have to be  $x$ . It could be  $y$  or  $z$ , or, even a string like `hello`.
2.  $\lambda x. M$ : a lambda abstraction, where  $x$  is a parameter variable and  $M$  is a term. This is how we define functions, which we call the process of "abstraction". Abstract in general math means "simple" or "minimal", so we're saying "I'm going to produce the result value  $M$  if you give me one additional piece of information" (which is  $x$ ).
3.  $MN$ : two terms next to each other means to pass the one on the right to the one on the left as the argument. (I say *the* instead of *a* because there is always exactly one parameter for every function)
4.  $(\lambda x. M)$  and  $(MN)$  are terms, too. You can use parentheses to control order.

# Lambda examples

$\lambda x. x + 1$  is a function that takes a number and adds 1 to it.

The one parameter is named  $x$ , and the one return value is  $1 +$  that.

We can mix lambda expresions and mathematical definitions. For example:

$$f = \lambda x. x + 1$$

$$8 = f \ 7$$

We have to agree on what operators like "+" mean to do this, though, which we'll talk about later (they aren't actually part of lambda calculus).

Before we move on, let's have some abstraction practice.

## Define these functions

1. A function that takes a value and doubles it.
2. A function that takes a value and ignores it and returns 0 instead.
3. A function that computes the positive square root of a value. This one doesn't have to be a lambda abstraction.

You can use ordinary math notation for the operations like doubling and square roots. You don't have to use numerical techniques or anything to get the square root.

# Abstraction answers

1.  $\lambda x. x + x$  or alternatively  $\lambda x. 2x$

2.  $\lambda x. 0$

3. sqrt

(it's already a function that takes one value and returns one value, but you can write it as an abstraction by saying  $\lambda x. \sqrt{x}$  and that's right, too)



## $\alpha$ conversion

Old-fashioned lambda calculus didn't have a notion of "scope". Variables just weren't supposed to conflict with each other.

There is a *reduction* we can perform to rename variables and ensure this. A reduction is just the process of taking a lambda term and simplifying or changing it.

In this case,  $\alpha$  conversion (read: "alpha conversion") means changing the name of the lambda parameter consistently throughout the whole term, as long as the new name does not conflict with another parameter.

It's like using the "rename" refactoring in your favorite IDE.

## $\alpha$ conversion (2)

For a simple lambda function with one parameter, we just change the name.

$\lambda x. x + x$  can become  $\lambda y. y + y$ , and

$\lambda x. x + x + x$  can become  $\lambda y. y + y + y$

But we can not use alpha conversion to turn  $\lambda x. x + y$  into  $\lambda y. y + y$ , because we were already using the variable  $y$  (from an external definition or outer parameter that I'm not showing)

And this is good, because  $\lambda x. x + y$  is a function that adds some external fixed value  $y$  to the given value  $x$ , whereas  $\lambda y. y + y$  just doubles the given value. They are completely different functions.

# $\alpha$ equivalence

If two lambda terms would be the same after an  $\alpha$  conversion, we call them alpha equivalent.

This is one way two lambda functions can be "equal". If they are literally the same function except their parameter is named differently, then we know they will meet the looser requirements for extrinsic functional equivalence (meaning, they will give the same output values for the same input values)

Functions can be equivalent without being  $\alpha$  equivalent, but if they are  $\alpha$  equivalent then you know they are equivalent.

## $\alpha$ practice

1. Perform an  $\alpha$  conversion to this term:  $\lambda x. x/2$
2. List an invalid  $\alpha$  conversion for this term:  $\lambda x. x/b$  and explain why it is invalid.
3. List a lambda term that is alpha equivalent to  $\lambda x. x + b$
4. Are  $\lambda x. x * y$  and  $\lambda z. z * y$  alpha equivalent? Why or why not?

## $\alpha$ answers

1.  $\lambda x. x/2$  can become  $\lambda q. q/2$
2.  $\lambda b. b/b$  would be an invalid  $\alpha$  conversion. It would change the function from dividing by  $b$  to the constant value 1 (or not a number if applied to 0).
3.  $\lambda \text{bloop}. \text{bloop} + b$  is  $\alpha$  equivalent to  $\lambda x. x + b$
4. Yes, because we can apply an alpha conversion from  $x$  to  $z$  without conflicting with any other variable.

# What about application

Rule 2 was abstraction. That just means defining a function.

Rule 3 is application. Suppose we define this function:  $\text{add7} = \lambda x. x + 7$

How do we use it? We just put the function next to the argument we want:

$$\text{add7 } 8 = 15$$

Lambda functions are *values*, so we can just substitute the actual definition for the name of the variable we put it in:

$$(\lambda x. x + 7) 8 = 15$$

This whole system is built around functions, so the easiest thing to do is call a function. You just write the function next to the value. There is no special syntax like  $f(7)$ , you just write  $f$  7. You *can* write  $f(7)$  if you, want, but the parentheses aren't needed.

# Application in detail

So how do we actually *do* the application? We apply a new reduction:  $\beta$  reduction.

$\beta$  reduction means replacing an abstraction's parameter with a given value. This means applying a single term to another term.

So for example:

$(\lambda x. x + 1) \ 7$  after beta reduction is  $(7 + 1)$

Notice that we delete the header of the term, (the  $\lambda x.$  part) and we replace every  $x$  with the argument 7.

So, to perform a single application, just do one round of  $\beta$  reduction.

# Normal form

If you keep applying until you can't anymore, you reach *normal form*.

For example:  $(\lambda x. x + y) 4$  is not in normal form, because we can apply the function.  
Normal form means we cannot reduce anymore.

We could apply the function to 4, and get this:

$$(\lambda x. x + y) 4 = 4 + y$$

We cannot reduce  $4 + y$  anymore, so now we're in normal form.



# Abstraction and application practice

1. Define a lambda function to triple a number and apply it to 7 in a single expression.
2. Define a lambda function to concatenate "llo" to the end of a string and apply it to "he" in a single expression. Use `++` as the concatenation operator.
3. Define a lambda function that always returns 0, and a lambda function that always returns 1, apply them to get those values, and add the results together.

# Abstraction and application answers

1.  $(\lambda x. 3 \cdot x) 7$

2.  $(\lambda s. s++\text{"llo"})\text{"he"}$

3.  $((\lambda a. 0) 7) + ((\lambda a. 1) 200) = 1$

# Function call?

Because imperative programming is based on changing state, when we *call* a function, we store the return address, jump to the function's address, and begin executing its instructions.

When we *apply* a function in lambda calculus, we just replace all instances of a term with a variable. Nothing actually *changes* outside the function.

This is not a coincidence. Functional programming doesn't *allow* you to change things outside of a function.

Questions?

# There is no way you only need one argument

I mean come on.

*Surely* we can't just only have functions of one argument? I mean, you used concatenation earlier! That takes two arguments!

You're right. We often need more than one argument. But it turns out, you can build functions of more than one argument out of functions with exactly one argument and this does not require bending the rules of reality.

It does require bending our brains a little, though...

# Currying

Suppose addition is defined, but we want to create a lambda term that does it.

We can write this:

$$\lambda x. \lambda y. x + y$$

This is a function that takes one argument, and returns a new function that takes another argument, that then returns the result of adding the two together.

This is confusing at first, so let's spend a moment to ponder it...

## Currying (2)

Let's try to add 7 to 8 using this lambda term. We write it like this:

$$(\lambda x. \lambda y. x + y) 7 8$$

Here, there are two arguments with a space between them. Is there a rule for that? No, it actually follows from  $\beta$  reduction.

Apply  $\beta$  reduction once. Replace every  $x$  with a 7:

$$(\lambda x. \lambda y. x + y) 7 8 = (\cancel{\lambda x.} \lambda y. 7 + y) 8$$

$$(\lambda y. 7 + y) 8 = \cancel{\lambda y.} 7 + 8 = 15$$

## Currying (3)

The process of representing a multi-parameter function as a series of single-parameter function definitions is called "currying", after American mathematician, [Haskell Curry](#).

Curry used it extensively in his work, but the original idea was originated by Soviet mathematician [Moses Schönfinkel](#) 6 years earlier, so sometimes people call it "Curry-Schönfinkeling".

Curry is so famous and influential, all three of his names were also used to name programming languages, including the one we're using in this class.



## Currying (4)

The big idea behind currying is that the outer function takes a parameter, which becomes *bound*, and returns a family of functions that use that parameter.

So if we stop at  $(\lambda x. \lambda y. x + y) \ 7$ , we get  $\lambda y. 7 + y$ , which is a function that adds 7 to things.

$(\lambda x. \lambda y. x + y)$  is a function that creates a function that adds  $x$  to whatever its argument is.

# Currying Practice

This can be a little challenging to understand, so let's do some drills:

1. Define a term that has 3 parameters and adds them together.
2. What is the result of partially applying that term on the values 7 and 8? (we say partially applying because we haven't supplied all 3 arguments).
3. Is it meaningful to only give that function 2 arguments? Why?
4. Suppose  $f = \lambda a \ b \ c. \dots$   
Should  $f \ 1 \ 2 \ 3$  be equivalent to  $f \ (1 \ (2 \ 3))$  or  $((f \ 1) \ 2) \ 3$  or is it something else?

## Currying answers

1.  $\lambda a. (\lambda b. (\lambda c. a + b + c))$  or equivalently,  $\lambda a. \lambda b. \lambda c. a + b + c$
2.  $(\lambda a. \lambda b. \lambda c. a + b + c) 7 8 = \lambda c. 7 + 8 + c = \lambda c. 15 + c$
3. Yes, it is meaningful because the result is a function that adds 15 to things.
4.  $f\ 1\ 2\ 3 \equiv ((f\ 1)\ 2)\ 3$ , and this makes sense because it means we don't need parentheses to pass multiple arguments. If it were the other way around, we'd be trying to call 2 on 3, instead of passing 2 as the second argument to  $f$ .

Note for #4: this means lambda application is *left associative*. That means we apply the arguments from left to right.

# Currying notation

Currying is so common, that there's special notation for it.

Instead of writing:

$$\lambda a. \lambda b. \lambda c. a + b + c$$

You can write:

$$\lambda abc. a + b + c$$

Those are equivalent. That's the main reason we have a `.` in the definition.

If you need long variable names, you can separate them with spaces:

$$\lambda \text{blip blorp blorp}. \text{blip} + \text{blorp} + \text{blorp}$$

**This is just shorthand.**  $\lambda abc. a + b + c$  is still just a 1 argument function that returns one result. It just so happens that that result is another function, etc.

Questions?

# Why did I have to learn that?

Because in many functional languages, including Haskell, every function takes one argument.

Any multi-argument function is typically written curried.

This allows some really cool tricks to be performed, although we'll have to wait to see them.

But just to give you an idea...

## Why did I have to learn that? (2)

If I want an increment function, I can write it like this:

```
inc = (1 +)
```

Which means "incrementing is the same as adding 1 to something".

Addition isn't special in Haskell. But it takes two arguments, and we're supplying the first one, so the result of doing that is a function that takes one argument and adds 1 to it. This is partial application in action, and currying makes it easy.

# The proof is in the Python

You might think "okay, this is some fancy functional stuff", but really it has more to do with the basic operation of anonymous functions / lambdas / closures.

You can do it in python:

```
f = lambda x: lambda y: lambda z: x + y + z
add15 = f(7)(8)
print(add15(9)) # prints 24
print(f(7)(8)(9)) # also prints 24
```

Really, you can do this in any language that supports anonymous closures. A closure is just a function that is allowed to use variables from its containing scope (environment).

(Note: I like to ask this kind of question on quizzes, so make sure you can do this in at least one real world programming language)



# Why haven't I seen that before?

The reason that snippet looks strange is that currying is not part of Python's culture. Python is an imperative language, and it prefers multi-parameter functions. Its "lambda" keyword can actually declare anonymous functions of many variables, so its name is somewhat misleading.

The reason we need several groups of parentheses is that we aren't calling one function on `7, 8, 9`, we're calling a series of functions:

```
a = f(7)
b = a(8)
c = b(9)
# c is now 24. a and b are lambda functions. Try printing f, a, and b.
```

But in functional languages, currying is the default way to have multiple parameters.

Questions?

# What does 7 mean?

I have been using expressions from general math that aren't actually lambda terms according to our definition.

Remember our definition of a lambda term? It's either a variable, a lambda abstraction, or an application, with or without parentheses.

Where is 7?

Where is anything for that matter?

# Is everything contained in lambda calculus?

In order for lambda calculus to be a model of computation, it can't require external symbols or operations.

Turing machines don't. Turing machines have a tape alphabet, but there's no requirement the things we write on the tape are even numbers. They could be completely made-up symbols.

And everything we do on a Turing machine is defined by the behavior of the machine. There is a set of turing states that will add two numbers together, so we don't need addition to be defined ahead of time.

So do we need to have an "alphabet" of built-in symbols and operations we can use?

# Defining stuff

Not exactly...

It turns out, in Lambda Calculus, everything is a lambda function.

*everything*

Even 7.

But let's start simple. Very simple. What about booleans?

# Bools in lambda

There are two boolean values: true and false.

There are many ways to encode them, but Church encoded them like this:

- $\text{true} = \lambda a. \lambda b. a$
- $\text{false} = \lambda a. \lambda b. b$

That is, both true and false are curried functions of two arguments. True returns its first argument, false returns its second argument.

And what are those arguments? They are also functions, but they can be whatever functions you want. True takes the first, false takes the second.

It's functions all the way down...

# Wait, where does it stop?

If "true" is a function, then that means we have to pass it a value. But what value should we use? Even our simplest boolean values are still functions!

We don't actually have to evaluate every function.

If you're writing a lambda program to compute a boolean function, it can return "true" as a function. You can then recognize that the return value is  $\lambda a. \lambda b. a$ , signifying a true result. You don't have to run it on anything.

It's kind of like how a turing machine can return a "screen", we just have to interpret the symbols on the tape as colors. Here's we're looking into the function to figure it out.

We could have defined true to be  $\lambda a. \lambda b. \lambda c. \lambda hi. a (b (c hi))$  if we wanted. (we don't)

But there are some nice properties of the definition we picked.

# Defining and

Now that we have:

- $\text{true} = \lambda a. \lambda b. a$
- $\text{false} = \lambda a. \lambda b. b$

How do we define and? That is, we want:

- $\text{and true true} = \text{true}$
- $\text{and true false} = \text{false}$
- $\text{and false true} = \text{false}$
- $\text{and false false} = \text{false}$

This is a puzzle. Any ideas?



## Defining and (2)

$\text{and} = \lambda a. \lambda b. a \ b \ \text{false}$

So there are two arguments,  $a$  and  $b$ . These are supposed to be booleans.

In this system, booleans are functions. They take two values.

So we pass  $b$  to the function  $a$ .

If  $a$  represents true, it returns its first argument,  $b$ .

If  $a$  represents false, it ignores  $b$  and returns its second argument, which is false.

Therefore, the only way to get true, is for  $a$  to be true, and when it returns  $b$ ,  $b$  must also be true.

# Worked boolean function practice

- Define 'or'
- Define 'not'
- Define 'xor'

# Boolean function answers

- $\text{or} = \lambda a\ b. a\ \text{true}\ b$

If  $a$  is true, just return true. Otherwise, return  $b$ .

(Also I'm using the short syntax for curried functions)

- $\text{not} = \lambda a. a\ \text{false}\ \text{true}$

If  $a$  is true, it returns false; if  $a$  is false it returns true. And vice versa.

- $\text{xor} = \lambda a\ b. a\ (\text{not}\ b)\ b$

If  $a$  is true,  $b$  must be false. If  $a$  is false,  $b$  must be true.

This works too, but is more verbose:

$\lambda a\ b. \text{and}\ (\text{or}\ a\ b)\ (\text{not}\ (\text{and}\ a\ b))$

But what about 'if'?

# What about 'if'?

True and false both behave kind of like little if-statements. If the function is true, it returns its first argument, and if false, returns its second.

But a true if-function is a 3 argument function. It takes:

- A condition
- A result if the condition is true
- A result if the condition is false

Any ideas?

## If in lambda

$\text{if} = \lambda c\ a\ b. c\ a\ b$

It takes 3 arguments, the condition and two terms.

If  $c$  is true, it will return  $a$ . If  $c$  is false, it will return  $b$ .

Questions?

# Natural number encoding

There are actually different ways to encode natural numbers. The most common are Church numerals (named after Alonzo Church).

If everything is a function, how do we represent numbers?

Well...with functions.

Specifically, a natural number is just a function that takes another function and a starting value and executes it a certain number of times!

Let's take a look...

# The encoding of 1

One is a function:  $\lambda \dots$

That takes a function and a starting value:  $\lambda f\ x. \dots$

And executes the function once:  $\lambda f\ x. f\ x$

That's...not very interesting.

But what about two?



# The encoding of 2 and 0

Now we define a function, that takes a function and a starting value, and executes that function twice.

This means, it calls the function on the result of calling the function on the starting value:

$$\lambda f\ x. f\ (f\ x)$$

Sometimes, computer scientists write this repeated application of the same function like this:  $f^2\ x$ . Note, this is different from  $(f\ x)^2$ , which is squaring/applying the result rather than the function.

And zero is encoded by just not calling the function and returning  $x$ :  $\lambda f\ x. x$

Quick knowledge check: [how do we encode 3?]

# Higher order functions

In lambda calculus (and functional programming in general), it is very common to make use of higher order functions.

A higher order function is a function that takes another function as an argument.

It can be hard to think about, but you do see this even in imperative languages. In C, for example, `qsort` takes a pointer to a function that is used to compare values in an array.

So a number is a higher order function. It is a function that takes another function (and a value), applies the other function a certain number of times.

## What about successor?

The successor to a natural number, unfortunately written "succ", is one more than it.

For example:  $\text{succ } 2 = 3$

How could we, given a number, increment it by one?

That is, how do we express succ as a lambda function?

# Defining the successor function

$\text{succ} = \lambda n. \lambda f x. f (n f x)$

Why? First we take a number, and then we *return* a function (which represents a new number). We take the given number, and "execute it" (which means applying  $f$   $n$  times starting on  $x$ ), and then run  $f$  one more time on the result of that.

The function that does this is the successor. It takes a number and returns another number that runs its function one more time.

# Using the syntactic sugar

In programming language design, syntactic sugar refers to making syntax nicer to express without changing the power of the language. In this case,  $\lambda n \ f \ x. \dots$  is syntactic sugar for  $\lambda n. \lambda f \ x. \dots$  (which itself is sugar for  $\lambda n. \lambda f. \lambda x. \dots$ )

We could also have written this:

$$\text{succ} = \lambda n \ f \ x. f \ (n \ f \ x)$$

Now it looks like a function that takes 3 arguments. *But nothing has changed.*

Writing it as a function that took a number and which returned a function of two arguments was just a way of notating it that expressed its purpose. The person using the function will typically only supply the first argument. But this was just a way of communicating, it did not change the function!

# Questions

# What about addition

If we treat  $a$  as meaning  $f^a x$  for some  $x$ , and  $b$  means  $f^b x$

What is  $a + b$ ?

How do we define it as a lambda function?

# Addition

What we really want is  $f^{(a+b)} x$ . Equivalently,  $f^a (f^b x)$

The parentheses are important here. We'll see why in a bit.

Addition looks like this:

$\text{plus} = \lambda a\ b. \lambda f\ x. a\ f\ (b\ f\ x)$

That is, first we apply  $f$  to  $x$   $b$  times (remember,  $b$  is a number, which means it is a function that takes another function and a value to repeatedly call it on)

Then we apply  $f$  to the result  $a$  times. So a total of  $a + b$  applications took place.

[Would it matter if we swapped the order of  $a$  and  $b$ ?]

What about multiplication?



# Multiplication

We want to add  $b$ ,  $a$  times. Or alternatively  $a$ ,  $b$  times.

Remember how numbers were just functions that apply another function?

What if the function we're applying is "add 5 to this"?

And we pass that function to the number 4, which represents "run this 4 times"?

Well, then we would be running "add 5 times" 4 times, which would be 4 times 5 (or 5 times 4).

$\text{times} = \lambda a \ b \ f \ x. a \ (b \ f) \ x$

Here, we're applying "run  $f$   $b$  times"  $a$  times, which has the effect of running  $f \ a \times b$  times.

## Multiplication (2)

Of course, we don't have to express our function with so much detail.

What if we wrote this?

$\text{times} = \lambda a\ b. a\ (\text{plus } b)\ 0$

This is actually equivalent, and nicer to read, too. It says "timesing is the same as taking two numbers,  $a$  and  $b$ , and applying 'plus  $b$ ' to 0,  $a$  times.

But it's starting to get complicated. It's not clear what variables refer to, anymore. 0 is actually a function. And plus is a function, and  $b$  is a function, but they're different kinds of function. They require different parameters.

We'll take a look at how to straighten this out in a second, but first, a strange question...

# Exponentiation

One way of defining it:

$$\text{exp} = \lambda a \, b \, f \, x. \, b \, a \, f \, x$$

notice the lack of parentheses. we're not calling  $a$  with  $f$  and  $x$ .

we're instead passing  $a$  as the function itself for  $b$ .

and we're passing  $f$  as the *value* for  $b$ . that is, we're doing this:

$$\text{exp} = \lambda a \, b \, f \, x. \, (b \, a) \, f \, x$$

So we're passing the "do something  $a$  times" function to  $b$ ...

## Exponentiation (2)

So imagine we call `exp 3 2`. This is what happens after beta reduction. Just replace  $a$  and  $b$  with 3 and 2:

$$\text{exp } 3 \ 2 = \lambda f \ x. (2 \ 3) \ f \ x$$

$$\text{And remember: } 2 = \lambda f \ x. f \ (f \ x) = \lambda f \ x. f^2 x$$

$$3 = \lambda f \ x. f \ (f \ (f \ x)) = \lambda f \ x. f^3 x$$

After substituting (pretend lexical scoping rules apply):

$$\text{exp } 3 \ 2 = \lambda f \ x. (2 \ 3) \ f \ x = (\lambda f \ x. f^2 x)(\lambda f \ x. f^3 x)$$

$$= (\lambda f \ x. f^3 x)^2 x$$

$$= \lambda f \ x. f^9 x$$

(if we apply  $f^3$  to itself twice, we end up with  $f^9$ , not  $f^6$ , because we're tripling twice)

## Exponentiation (3)

But, again, we can just use our earlier building blocks to make exponentiation.

$\text{exp } a \ b = b \ (\text{times } a) \ 1$

We're just applying "multiply by  $a$ ",  $b$  times

But what is the "1" for?

It's the starting value. We're going to run "times  $a$ "  $b$  times starting with the value 1.

So we will get:  $f^9 1$

But remember, 1 itself is a function that takes a function and a value.

We aren't specifying those values, so "exp  $a \ b$ " is actually a proper number. That is, it's a function that takes two values.

# Exponentiation (\$)

Compare to: times  $a \ b = b$  (plus  $a$ ) 0

"call 'plus a' b times, starting with a value of 0"

"call 'times a' b times, starting with a value of 1"

We're starting to see how smaller lambda abstractions can build into bigger ones.

Unlike turing machines, which are kind of awkward to work with unless you show the equivalence of RAM and registers to tape, lambda calculus can just keep scaling.

# Function calling notation

One last thing, I used this notation:

$\text{exp } a \ b = \dots$

That's equivalent to:

$\text{exp} = \lambda a \ b. \dots$

The difference is just notational.

# Questions?

This is rough stuff, I know.



# Types

We've been saying things like "a and b are numbers", but what even is a number?

It's a function, right?

Is it possible to use it *wrong*? Like, what if something is expecting a function.

Is it even *possible* for something to not be a function?

# Not really

In lambda calculus, literally everything is a function.

Classic lambda calculus is called "untyped lambda calculus".

The simplest possible value is:  $\text{id} = \lambda x. x$ , the identity function.

It takes one argument, and it returns it. So what is "id id id"?

Is it possible to write a bad lambda expressions? At least, when all the variables are bound?

## Again, not really

It's just "id". Because  $\text{id id id} = (\text{id id}) \text{id} = \text{id id} = \text{id}$ .

We can absolutely write bugs in lambda calculus. We can also write infinite loops like this one:  $(\lambda x. x x)(\lambda x. x x)$ .

But type errors we cannot write. Because there is only one type, and no way to produce anything else.

# Is that good?

This seems like a cause for celebration, but it's not.

Sometimes we have to interact with things outside our pristine lambda program.

Suppose we eventually want to run machine code. Now we want to store "ints".

At this precise point, we have to care. Because if  $x$  is an int, it becomes wrong to say  $(x\ x)$ . You can't "call" an int with anything, so this expression makes no sense.

And that's really the point to all this. Believe it or not, lambda calculus isn't just a theoretical thing. It's actually "assembly language" for functional programming, and we need some way to connect it to the real world.

And types give us a way to make sure we're doing that correctly.

# Basic types

There is a whole syntax for typed lambda terms, but we aren't going to learn it.

Just be aware that there is a way to say "in this lambda term,  $x$  is an int and not a function or anything else."

We'll see how Haskell handles types next module.

# The point

We have just learned an arcane way of expressing computation. Even more arcane than turing machines. We have learned confusing symbols, Greek letters, and some complex mathematical re-definitions.

Why?

Because this is fundamentally how functional programming works.

Many functional programming languages are convenient wrappers around, effectively, lambda calculus.

They also use the underlying machine more effectively. 8 will be an integer in a register, not a function applied 8 times (although it's interesting to know it could be).

## The point (2)

Haskell, the main language of this class, has an intermediate language called *Core*.

*Core* is basically lambda calculus, plus types, plus basic syntax for binding expressions to names. It's Haskell minus advanced features.

And it turns out, it's kind of a subset. That is, Haskell supports lambda calculus out of the box. It is useful.

And the features that Haskell provides are pretty minor syntactic convenience. Most code will be functions, and all functions are lambda functions.

# What is plus times plus?

I *highly* recommend watching [this video](#), to give you a deeper appreciation for all this lambda calculus.

It also teaches a [cool notation](#) for lambda terms that look like something that aliens carved into a rock 20,000 years ago.



Questions?

# The quiz

We will have our first practice quiz on [insert date here].

(I will put the actual date on Canvas).

This quiz will be 15 minutes if you take it in class.

You can bring any written or printed material into the exam, but no electronic devices (except to submit at the very end--when you've put away your pens or pencils).

For extended time or other accommodations, you must take the quiz at the access center.

## The quiz (2)

When studying, be sure to spend time studying under a time limit.

As you take practice quizzes, you will notice the time it takes you go down. That's the grind working!

I'm going to give several practice quizzes here. For more, you can create your own lambda expressions, or alternatively, ask an AI to generate more quizzes and grade them. This is a way to use AI as a good learning tool.

# Practice Quiz 1

Show/explain work for partial credit.

1. (10%) Reduce  $(\lambda x y. x) z$  to normal form.
2. (20%) Reduce  $((\lambda x y. y x) w) (\lambda z. z)$  to normal form.
3. (20%) Define a curried function that computes  $2 * a + 3 * b + c$  for some values  $a, b, c$ . You can use the arithmetic operator "+" and multiplication operator "\*".
4. (20%) What is 6 expressed as a church numeral? Show the full lambda function.
5. (30%) Let  $\text{true} = \lambda a b. a$  and  $\text{false} = \lambda a b. b$ .

Define  $\text{nand } a \ b = \text{not } (\text{and } a \ b)$  as a lambda function (your final answer should only include pure lambda terms and 'true' and 'false, but intermediate steps can include the definitions of boolean functions we have learned)

## Practice Quiz 1 answers

1.  $\lambda y. z$ . We replaced  $x$  with  $z$  and removed one layer of abstraction.
2.  $((\lambda x y. y x) w) (\lambda z. z) = ((\lambda y. y w) (\lambda z. z)) = (\lambda z. z) w = w$
3.  $\lambda a. \lambda b. \lambda c. 2 * a + 3 * b + c$   
 $\lambda a b c. 2 * a + 3 * b + c$  is also correct.
4.  $\lambda f x. f (f (f (f (f x))))$
5.  $\text{nand } a b = a (b \text{ false true}) \text{ true}$

## Practice quiz 2

1. (10%) Reduce  $(\lambda x. \lambda y. y) (\lambda z. z)$  to normal form.
2. (20%) Reduce  $(\lambda f. f (f a)) (\lambda g. g b)$  to normal form.
3. (20%) Define a curried function that computes  $a^2 + b^2 + c^2$  for some values  $a, b, c$ . You may use arithmetic operators  $+$  and  $*$ .
4. (20%) Define a function that takes 2 numbers and returns 5 if they are equal, otherwise returning 7. You may use `==` as an equality operator returning a Church bool. Otherwise, use only lambda terms (no assuming that `if` is defined).
5. (30%) Let  $\text{true} = \lambda a b. a$  and  $\text{false} = \lambda a b. b$ . Define  $\text{xor } a b = \text{or } (\text{and } a (\text{not } b)) (\text{and } b (\text{not } a))$  as a lambda function (only include pure lambda terms and 'true' and 'false, but intermediate steps can include the definitions of boolean functions we have learned)

## Practice quiz 2 answers

1.  $(\lambda y. y)$  (the  $x$  isn't used anywhere, so the argument disappears)
2.
  - i.  $(\lambda f. f (f a)) (\lambda g. g b)$
  - ii.  $(\lambda g. g b)((\lambda g. g b) a)$
  - iii.  $((\lambda g. g b) a) b$
  - iv.  $(a b) b$
3.  $\lambda a b c. a * a + b * b + c * c$
4.  $\lambda a b. (a == b) 5 7$
5.  $\text{xor } a b = a (b \text{ false true}) (b \text{ true false})$

## Practice quiz 3 (unworked)

1. (10%) Reduce  $(\lambda x. x\ x)\ (\lambda y. y)$  to normal form.
2. (20%) Reduce  $(\lambda x. \lambda y. x\ y)\ (\lambda z. z + 1)\ 5$  to normal form. (you can treat numbers and `+` as built in arithmetic)
3. (20%) Define a curried function in a real language that computes  $a + 2 * b$
4. (30%) Using Church booleans, define an if-elseif-else function that takes one condition for the if, a value if that condition is true, a condition for the else if, a value for when that condition is true, and then a value for when neither condition is true.
5. (20%) Let  $\text{true} = \lambda a\ b. a$  and  $\text{false} = \lambda a\ b. b$ . Define the function *implies*, where  $\text{implies true true} = \text{true}$ ,  $\text{implies true false} = \text{false}$ ,  $\text{implies false true} = \text{true}$ , and  $\text{implies false false} = \text{true}$ .



# Ask an AI

Ask an AI to generate a practice quiz for you like the above! Give it the slides starting with "# Quiz Format" and up to and including this slide.

Then, ask it to grade you. I like to use this scale:

1. 0 points off for extremely minor things. Misspellings or missing grouping operators that are clearly intended.
2. 5 points for mistakes that cause the code to fail but are more than just minor mistakes. For example a small type error where it's clear you get the big idea but, e.g., applied the applicative to too many arguments or something.
3. 10 points for bigger mistakes, like type errors that can't work, but there's still "more than half" of the understanding demonstrated.
4. Zero points total if there are several major mistakes or it looks like you're guessing.

# Required reading

In addition to studying for the quiz, make sure you're keeping up on your reading.

We're assuming you already read the first two chapters of the Haskell Wikibook.

Now, work through:

- [Getting Set Up](#) and [Variables and Functions](#)
- [Truth Values](#) to learn about boolean functions.
- [Type Basics](#) to learn how to describe types.
- [Lists and Tuples](#) to learn how to describe lists and tuples.
- [Next steps](#) to learn control structures.
- [Building vocabulary](#) to learn about composition

These chapters aren't too long, but what's really important is the thinking afterward.

Questions?